Market Kurly
Database Design

마켓컬리 주문 및 배송 관리 시스템

회원 · 상품 · 주문 · 배송 데이터를 Oracle SQL 로 설계한
온라인 장보기 서비스 흐름 분석 프로젝트

Oracle SQL Project

팀명 : 장보러가조
팀원 : 박슬지 , 민재홍 , 장영훈 , 이남규 , 손지혜



목차

프로젝트 발표 흐름

01 프로젝트 개요

02 팀 소개 및 역할

03 요구사항 명세서

04 ERD 및 테이블 설계

05 SQL 구현 코드

06 전체 데이터 INSERT

07 질의문 및 프로시저

08 개인별 평가 소감

Fresh Data Flow | 회원에서 배송까지 실제 데이터 전체를 기준으로 설명

프로젝트 개요

마켓컬리의 주문·배송 흐름을 데이터베이스로 표현

항목	내용
프로젝트명	마켓컬리 주문 및 배송 관리 시스템
개발 목적	회원·상품·주문·배송 데이터를 Oracle SQL로 설계하고 실제 쇼핑몰 흐름처럼 관리
구현 환경	Oracle SQL Developer
주요 기능	회원가입 → 상품 조회 → 주문 생성 → 주문상세 저장 → 배송 상태 관리
구현 범위	테이블 6개 · PRIMARY KEY 6개 · FOREIGN KEY 5개 · 기초 질의문 10개 이상 · 프로시저 1개

발표 핵심: 요구사항 명세서와 ERD를 중심으로 데이터 흐름을 설명합니다.

팀 소개 및 역할 분담

팀원별 담당 업무와 산출물

팀원	담당 업무	산출물
박슬지	요구사항 분석	요구사항 명세서
민재홍	ERD 설계 / 발표	테이블 관계도, PK/FK 정리, 발표 대본
장영훈	프로시저 / PPT 정리	프로시저, 서브쿼리, 최종 PPT
이남규	테이블 생성 / 데이터 삽입	CREATE, INSERT SQL
손지혜	질의문 작성	기초 질의문, JOIN, GROUP BY

역할은 나누되, 최종 코드와 실행 결과는 팀 전체가 함께 검토했습니다.

프로젝트 수행 절차

기획 → 설계 → 구현 → 검증 → 발표

단계	수행 내용	결과물
1	주제 선정	마켓컬리 주문·배송 관리 DB 선정
2	요구사항 분석	회원·카테고리·상품·주문·주문상세·배송 기능 도출
3	ERD 작성	PK, FK, 1:N / 1:1 관계 정의
4	테이블 구현	CREATE TABLE 및 제약조건 작성
5	데이터 입력	회원 10명, 상품 50개, 주문·배송 데이터 입력
6	질의문·프로시저	SELECT, JOIN, GROUP BY, 서브쿼리, 재고 추가 프로시저
7	검증 및 발표	실행 결과 확인, PPT 정리

다음 프로젝트 보완: 수행 절차는 날짜별로 정리하여 진행 과정을 더 명확하게 표현할 예정입니다.

요구사항 명세서

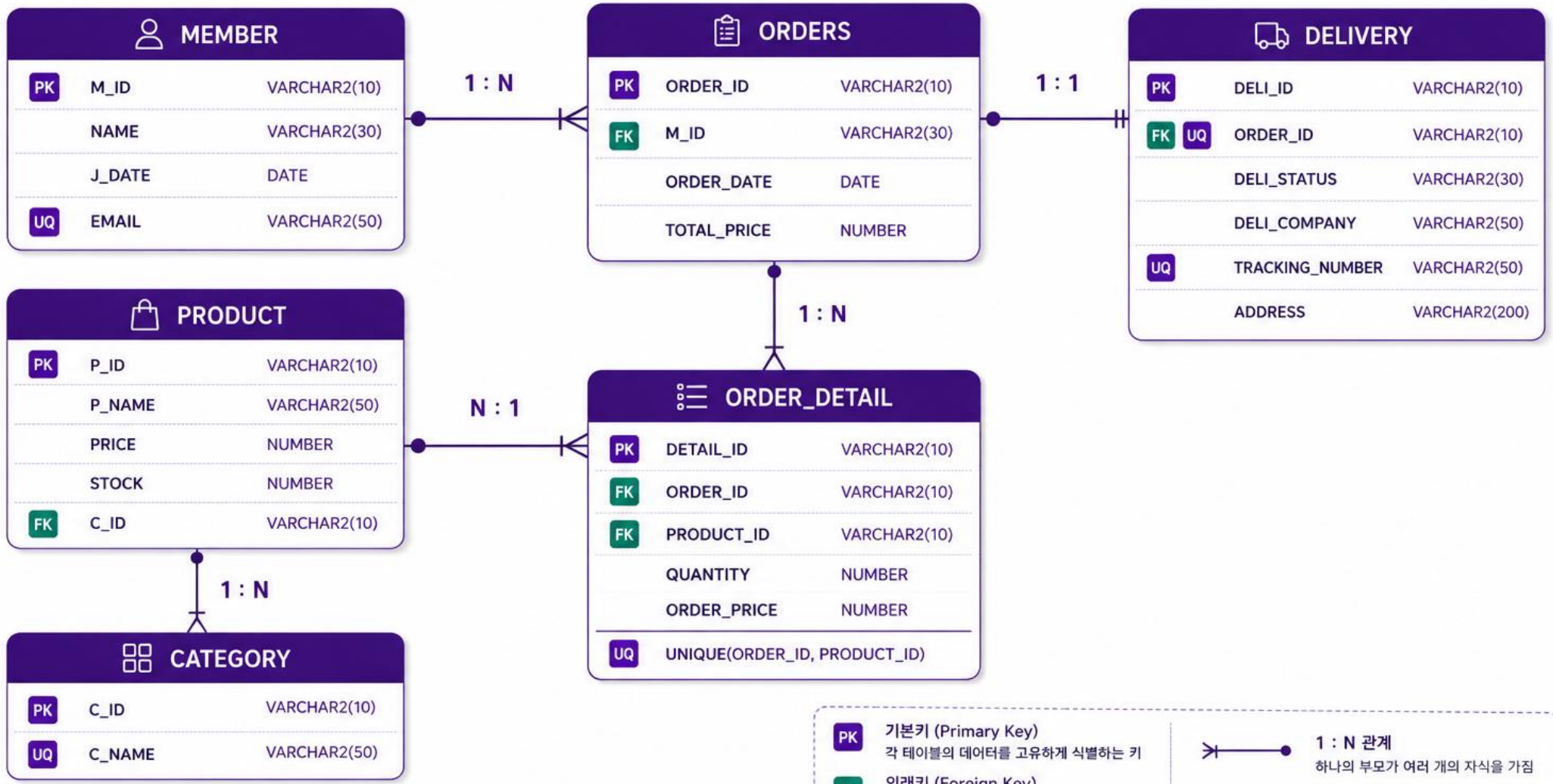
역할 기준: 사용자 기능 · 관리자 기능

No	역할	구분	기능명	요구사항 설명
1	사용자	회원 (MEMBER)	회원가입	• 회원번호(M_ID), 이름(NAME), 가입일(J_DATE), 이메일(EMAIL)을 저장한다. • M_ID는 PK이며, NAME·EMAIL은 필수 입력, EMAIL은 중복 금지이다. • 가입일은 가입이 완료된 날짜로 자동 저장된다.
2	사용자	주문 (ORDERS)	상품 주문	• 결제가 완료되면 주문번호(ORDER_ID), 회원번호(M_ID), 주문일(ORDER_DATE), 총금액 (TOTAL_PRICE)을 저장한다. • ORDER_ID는 PK, M_ID는 MEMBER를 참조하는 FK이며 주문일은 결제 완료 날짜이다.
3	사용자	주문상세 (ORDER_DETAIL)	주문 상세 등록	• 주문번호, 상품번호, 수량(QUANTITY), 상품별 금액(ORDER_PRICE)을 기록한다. • DETAIL_ID는 PK, ORDER_ID·PRODUCT_ID는 FK이며 같은 주문·상품 조합의 중복을 막는다.
4	관리자	카테고리 (CATEGORY)	카테고리 관리	• 카테고리번호(C_ID)와 카테고리명(C_NAME)을 등록한다. • C_ID는 PK, C_NAME은 필수 입력·중복 금지이며 상품 분류 기준으로 사용한다.
5	관리자	상품 (PRODUCT)	상품 등록 및 관리	• 상품번호(P_ID), 상품명(P_NAME), 가격(PRICE), 재고(STOCK), 카테고리번호(C_ID)를 저장한다. • P_ID는 PK, C_ID는 CATEGORY를 참조하는 FK이며 가격은 0원 초과, 재고는 0개 이상이다.
6	관리자	배송 (DELIVERY)	배송 관리	• 배송상태, 배송회사, 운송장번호, 배송주소를 등록하고 관리한다. • DELI_ID는 PK, ORDER_ID는 ORDERS를 참조하는 FK·UNIQUE이며 운송장번호도 중복될 수 없다. • 배송상태는 배송준비중·배송중·배송완료 중 하나로 저장한다.

핵심 흐름	회원가입 → 상품 조회·선택 → 결제 완료 및 주문 확정 → 주문·주문상세 저장 → 배송정보 연결

마켓컬리 주문·배송 관리 ERD

회원 · 상품 · 주문 · 배송 테이블 관계도



PK 기본키 (Primary Key)
각 테이블의 데이터를 고유하게 식별하는 키

FK 외래키 (Foreign Key)
다른 테이블의 기본키를 참조하는 키

UQ 유니크 (Unique)
중복 값을 가질 수 없는 제약조건

1 : N 관계
하나의 부모가 여러 개의 자식을 가짐

1 : 1 관계
하나의 부모가 하나의 자식을 가짐

※ ORDERS.ORDER_DATE는 장바구니에 담은 날짜가 아니라, 결제를 완료하여 주문이 확정된 날짜를 의미합니다.

※ ORDER_DETAIL의 금액을 모두 합하면 ORDERS의 TOTAL_PRICE가 됩니다.

테이블 설계 요약

6개 테이블로 마켓컬리 주문 흐름을 구성

테이블	핵심 컬럼	PK / FK	역할
MEMBER	M_ID, NAME, J_DATE, EMAIL	PK: M_ID	회원 정보 관리
CATEGORY	C_ID, C_NAME	PK: C_ID	상품 분류 관리
PRODUCT	P_ID, P_NAME, PRICE, STOCK, C_ID	PK: P_ID / FK: C_ID	판매 상품 및 재고 관리
ORDERS	ORDER_ID, M_ID, ORDER_DATE, TOTAL_PRICE	PK: ORDER_ID / FK: M_ID	결제 완료 주문 정보 저장
ORDER_DETAIL	DETAIL_ID, ORDER_ID, PRODUCT_ID, QUANTITY, ORDER_PRICE	PK: DETAIL_ID / FK: ORDER_ID, PRODUCT_ID	주문한 상품과 수량 저장
DELIVERY	DELI_ID, ORDER_ID, DELI_STATUS, DELI_COMPANY, TRACKING_NUMBER, ADDRESS	PK: DELI_ID / FK: ORDER_ID	배송 상태 및 운송장 관리

테이블 생성 코드 설명 1

MEMBER · CATEGORY · PRODUCT

테이블 생성 ① MEMBER · CATEGORY · PRODUCT

```
01  CREATE TABLE MEMBER(
02    M_ID VARCHAR2(30) PRIMARY KEY,
03    NAME VARCHAR2(50) NOT NULL,
04    J_DATE DATE DEFAULT SYSDATE NOT NULL,
05    EMAIL VARCHAR2(50) NOT NULL UNIQUE
06  );
07
08  CREATE TABLE CATEGORY(
09    C_ID VARCHAR2(10) PRIMARY KEY,
10    C_NAME VARCHAR2(50) NOT NULL UNIQUE
11  );
12
13  CREATE TABLE PRODUCT(
14    P_ID VARCHAR2(10) PRIMARY KEY,
15    P_NAME VARCHAR2(50) NOT NULL,
16    PRICE NUMBER NOT NULL CHECK(PRICE > 0),
17    STOCK NUMBER DEFAULT 0 NOT NULL CHECK(STOCK >= 0),
18    C_ID VARCHAR2(10) NOT NULL,
19    CONSTRAINT FK_PRODUCT_CATEGORY
20      FOREIGN KEY(C_ID) REFERENCES CATEGORY(C_ID)
21  );
```

테이블	주요 컬럼	제약조건 / 의미
MEMBER	M_ID, NAME, J_DATE, EMAIL	M_ID PK, EMAIL UNIQUE, 가입일 기본값 SYSDATE
CATEGORY	C_ID, C_NAME	C_ID PK, C_NAME UNIQUE
PRODUCT	P_ID, P_NAME, PRICE, STOCK, C_ID	P_ID PK, C_ID FK, PRICE>0, STOCK>=0

기본 정보 테이블: 회원과 카테고리를 먼저 만들고, PRODUCT가 CATEGORY를 참조합니다.

테이블 생성 코드 설명 2

ORDERS · ORDER_DETAIL · DELIVERY

테이블 생성 ② ORDERS · ORDER_DETAIL · DELIVERY

```
CREATE TABLE ORDERS(
  ORDER_ID VARCHAR2(10) PRIMARY KEY,
  M_ID VARCHAR2(30) NOT NULL,
  ORDER_DATE DATE DEFAULT SYSDATE NOT NULL,
  TOTAL_PRICE NUMBER NOT NULL CHECK(TOTAL_PRICE >= 0),
  CONSTRAINT FK_ORDERS_MEMBER FOREIGN KEY(M_ID) REFERENCES MEMBER(M_ID)
);

CREATE TABLE ORDER_DETAIL(
  DETAIL_ID VARCHAR2(10) PRIMARY KEY,
  ORDER_ID VARCHAR2(10) NOT NULL,
  PRODUCT_ID VARCHAR2(10) NOT NULL,
  QUANTITY NUMBER NOT NULL CHECK(QUANTITY > 0),
  ORDER_PRICE NUMBER NOT NULL CHECK(ORDER_PRICE > 0),
  CONSTRAINT FK_DETAIL_ORDERS FOREIGN KEY(ORDER_ID) REFERENCES ORDERS(ORDER_ID),
  CONSTRAINT FK_DETAIL_PRODUCT FOREIGN KEY(PRODUCT_ID) REFERENCES PRODUCT(P_ID),
  UNIQUE(ORDER_ID, PRODUCT_ID)
);

CREATE TABLE DELIVERY(
  DELI_ID VARCHAR2(10) PRIMARY KEY,
  ORDER_ID VARCHAR2(10) NOT NULL UNIQUE,
  DELI_STATUS VARCHAR2(30) NOT NULL CHECK(DELI_STATUS IN ('배송준비중', '배송중', '배송완료')),
  DELI_COMPANY VARCHAR2(50) NOT NULL,
  TRACKING_NUMBER VARCHAR2(50) NOT NULL UNIQUE,
  ADDRESS VARCHAR2(200) NOT NULL,
  CONSTRAINT FK_DELIVERY_ORDER FOREIGN KEY(ORDER_ID) REFERENCES ORDERS(ORDER_ID)
);
```

테이블	주요 컬럼	제약조건 / 의미
ORDERS	ORDER_ID, M_ID, ORDER_DATE, TOTAL_PRICE	ORDER_ID PK, M_ID FK, 결제 완료 주문 저장
ORDER_DETAIL	DETAIL_ID, ORDER_ID, PRODUCT_ID, QUANTITY, ORDER_PRICE	ORDER_ID·PRODUCT_ID FK, 같은 주문·상품 중복 방지
DELIVERY	DELI_ID, ORDER_ID, DELI_STATUS, TRACKING_NUMBER, ADDRESS	ORDER_ID FK·UNIQUE, 운송장번호 UNIQUE

주문 흐름 테이블: ORDERS를 기준으로 ORDER_DETAIL과 DELIVERY가 각각 연결됩니다.

데이터 삽입 ① MEMBER · CATEGORY

```sql
-- MEMBER 데이터 10개
INSERT INTO MEMBER VALUES('M001', '김민준', DATE '2026-01-12', 'minjun3841@naver.com');
INSERT INTO MEMBER VALUES('M002', '이서연', DATE '2026-01-31', 'seoyeon0228@naver.com');
INSERT INTO MEMBER VALUES('M003', '박지훈', DATE '2026-02-18', 'jihoon.park@gmail.com');
INSERT INTO MEMBER VALUES('M004', '최유진', DATE '2026-03-01', 'yujinchoi123@naver.com');
INSERT INTO MEMBER VALUES('M005', '정현우', DATE '2026-03-25', 'hyunwoo0603@gmail.com');
INSERT INTO MEMBER VALUES('M006', '한지민', DATE '2026-04-20', 'hanjm@gmail.com');
INSERT INTO MEMBER VALUES('M007', '강도윤', DATE '2026-05-10', 'dotaek07@naver.com');
INSERT INTO MEMBER VALUES('M008', '윤수아', DATE '2026-05-30', 'sua4701@naver.com');
INSERT INTO MEMBER VALUES('M009', '임재현', DATE '2026-06-28', 'jaehyun2@naver.com');
INSERT INTO MEMBER VALUES('M010', '송하은', DATE '2026-07-01', 'haeun.song1221@naver.com');

-- CATEGORY 데이터 5개
INSERT INTO CATEGORY VALUES('C001', '채소');
INSERT INTO CATEGORY VALUES('C002', '수산, 해산, 건어물');
INSERT INTO CATEGORY VALUES('C003', '정육, 가공육, 달걀');
INSERT INTO CATEGORY VALUES('C004', '과일, 견과, 쌀');
INSERT INTO CATEGORY VALUES('C005', '간식, 과자, 떡');
```

데이터 삽입 ② PRODUCT P001 ~ P020

```sql
-- PRODUCT 데이터 ① 채소 / 수산·해산·건어물
INSERT INTO PRODUCT VALUES('P001', '양파 1.5kg', 4990, 120, 'C001');
INSERT INTO PRODUCT VALUES('P002', '오이 3입', 4990, 80, 'C001');
INSERT INTO PRODUCT VALUES('P003', '고구마 1kg', 9990, 60, 'C001');
INSERT INTO PRODUCT VALUES('P004', '감자 1kg', 4990, 150, 'C001');
INSERT INTO PRODUCT VALUES('P005', '대파 500g', 4490, 90, 'C001');
INSERT INTO PRODUCT VALUES('P006', '당근', 6490, 75, 'C001');
INSERT INTO PRODUCT VALUES('P007', '알배기 배추 1봉', 3990, 45, 'C001');
INSERT INTO PRODUCT VALUES('P008', '친환경 표고버섯 300g', 5990, 35, 'C001');
INSERT INTO PRODUCT VALUES('P009', '양배추 1.5kg', 4990, 65, 'C001');
INSERT INTO PRODUCT VALUES('P010', '무농약 콩나물 300g', 1590, 180, 'C001');
INSERT INTO PRODUCT VALUES('P011', '국산 오징어', 14900, 30, 'C002');
INSERT INTO PRODUCT VALUES('P012', '생연어', 15900, 25, 'C002');
INSERT INTO PRODUCT VALUES('P013', '고등어(냉동)', 11900, 40, 'C002');
INSERT INTO PRODUCT VALUES('P014', '낙지 볶음', 9900, 20, 'C002');
INSERT INTO PRODUCT VALUES('P015', '오징어 볶음', 9900, 18, 'C002');
INSERT INTO PRODUCT VALUES('P016', '백명란', 12700, 28, 'C002');
INSERT INTO PRODUCT VALUES('P017', '생새우살', 8400, 35, 'C002');
INSERT INTO PRODUCT VALUES('P018', '완도 전복', 15800, 22, 'C002');
INSERT INTO PRODUCT VALUES('P019', '양념 장어', 31800, 12, 'C002');
INSERT INTO PRODUCT VALUES('P020', '갓구운 김 16봉', 5890, 85, 'C002');
```

데이터 삽입 ③ PRODUCT P021 ~ P035

```sql
-- PRODUCT 데이터 ② 정육·가공육·달걀 / 과일 일부
INSERT INTO PRODUCT VALUES('P021', '양념 소불고기', 15200, 25, 'C003');
INSERT INTO PRODUCT VALUES('P022', '유정란 20구', 10840, 70, 'C003');
INSERT INTO PRODUCT VALUES('P023', '삼겹살 구이용 600g', 21200, 40, 'C003');
INSERT INTO PRODUCT VALUES('P024', '대패삼겹살 1kg', 16990, 32, 'C003');
INSERT INTO PRODUCT VALUES('P025', '쪽갈비', 15600, 20, 'C003');
INSERT INTO PRODUCT VALUES('P026', '떡갈비', 9900, 55, 'C003');
INSERT INTO PRODUCT VALUES('P027', '훈제오리 300g', 29900, 18, 'C003');
INSERT INTO PRODUCT VALUES('P028', '한돈 목살 구이용 600g', 21200, 24, 'C003');
INSERT INTO PRODUCT VALUES('P029', '꽃삼겹살 구이용 600g', 21000, 21, 'C003');
INSERT INTO PRODUCT VALUES('P030', 'LA 갈비 500g', 22900, 15, 'C003');
INSERT INTO PRODUCT VALUES('P031', '수박 6kg', 34900, 18, 'C004');
INSERT INTO PRODUCT VALUES('P032', '해남 쌀 10kg', 39900, 50, 'C004');
INSERT INTO PRODUCT VALUES('P033', '방울토마토 750g', 9990, 48, 'C004');
INSERT INTO PRODUCT VALUES('P034', '사과 4입', 8990, 55, 'C004');
INSERT INTO PRODUCT VALUES('P035', '자두 1.2kg', 18900, 20, 'C004');
```

데이터 삽입 ④ PRODUCT P036 ~ P050

```sql
-- PRODUCT 데이터 ③ 과일·견과·쌀 / 간식·과자·떡
INSERT INTO PRODUCT VALUES('P036', '블루베리 300g', 8900, 35, 'C004');
INSERT INTO PRODUCT VALUES('P037', '골드키위 6입', 10900, 30, 'C004');
INSERT INTO PRODUCT VALUES('P038', '오렌지 6입', 9900, 40, 'C004');
INSERT INTO PRODUCT VALUES('P039', '레몬 4입', 5900, 45, 'C004');
INSERT INTO PRODUCT VALUES('P040', '생체리 300g', 10900, 25, 'C004');
INSERT INTO PRODUCT VALUES('P041', '초코칩 쿠키', 3980, 90, 'C005');
INSERT INTO PRODUCT VALUES('P042', '허니버터 아몬드', 5990, 70, 'C005');
INSERT INTO PRODUCT VALUES('P043', '감자칩 오리지널', 2480, 120, 'C005');
INSERT INTO PRODUCT VALUES('P044', '새우깡', 1480, 150, 'C005');
INSERT INTO PRODUCT VALUES('P045', '찰떡 500g', 7900, 35, 'C005');
INSERT INTO PRODUCT VALUES('P046', '인절미 400g', 6900, 30, 'C005');
INSERT INTO PRODUCT VALUES('P047', '약과 세트', 9900, 28, 'C005');
INSERT INTO PRODUCT VALUES('P048', '찹쌀 모찌', 8900, 45, 'C005');
INSERT INTO PRODUCT VALUES('P049', '크림치즈 베이글칩', 4980, 60, 'C005');
INSERT INTO PRODUCT VALUES('P050', '쌀과자', 3480, 80, 'C005');
```

데이터 삽입 ⑤ ORDERS

```sql
-- ORDERS 데이터 10개
INSERT INTO ORDERS VALUES('O001', 'M001', DATE '2026-06-30', 47520);
INSERT INTO ORDERS VALUES('O002', 'M003', DATE '2026-07-02', 74580);
INSERT INTO ORDERS VALUES('O003', 'M010', DATE '2026-07-02', 65150);
INSERT INTO ORDERS VALUES('O004', 'M007', DATE '2026-07-05', 14340);
INSERT INTO ORDERS VALUES('O005', 'M008', DATE '2026-07-08', 53180);
INSERT INTO ORDERS VALUES('O006', 'M009', DATE '2026-07-09', 34900);
INSERT INTO ORDERS VALUES('O007', 'M002', DATE '2026-07-11', 32460);
INSERT INTO ORDERS VALUES('O008', 'M004', DATE '2026-07-11', 23580);
INSERT INTO ORDERS VALUES('O009', 'M005', DATE '2026-07-13', 36100);
INSERT INTO ORDERS VALUES('O010', 'M006', DATE '2026-07-15', 24700);
```

데이터 삽입 ⑥ ORDER_DETAIL OD001 ~ OD018

```
-- ORDER_DETAIL 데이터 ① O001 ~ O003
INSERT INTO ORDER_DETAIL VALUES('OD001', 'O001', 'P003', 1, 9990);
INSERT INTO ORDER_DETAIL VALUES('OD002', 'O001', 'P020', 1, 5890);
INSERT INTO ORDER_DETAIL VALUES('OD003', 'O001', 'P026', 1, 9900);
INSERT INTO ORDER_DETAIL VALUES('OD004', 'O001', 'P022', 1, 10840);
INSERT INTO ORDER_DETAIL VALUES('OD005', 'O001', 'P040', 1, 10900);
INSERT INTO ORDER_DETAIL VALUES('OD006', 'O002', 'P002', 1, 4990);
INSERT INTO ORDER_DETAIL VALUES('OD007', 'O002', 'P007', 1, 3990);
INSERT INTO ORDER_DETAIL VALUES('OD008', 'O002', 'P012', 1, 15900);
INSERT INTO ORDER_DETAIL VALUES('OD009', 'O002', 'P027', 1, 29900);
INSERT INTO ORDER_DETAIL VALUES('OD010', 'O002', 'P036', 1, 8900);
INSERT INTO ORDER_DETAIL VALUES('OD011', 'O002', 'P037', 1, 10900);
INSERT INTO ORDER_DETAIL VALUES('OD012', 'O003', 'P004', 1, 4990);
INSERT INTO ORDER_DETAIL VALUES('OD013', 'O003', 'P005', 1, 4490);
INSERT INTO ORDER_DETAIL VALUES('OD014', 'O003', 'P014', 1, 9900);
INSERT INTO ORDER_DETAIL VALUES('OD015', 'O003', 'P030', 1, 22900);
INSERT INTO ORDER_DETAIL VALUES('OD016', 'O003', 'P033', 1, 9990);
INSERT INTO ORDER_DETAIL VALUES('OD017', 'O003', 'P041', 1, 3980);
INSERT INTO ORDER_DETAIL VALUES('OD018', 'O003', 'P048', 1, 8900);
```

데이터 삽입 ⑦ ORDER_DETAIL OD019 ~ OD032

```
-- ORDER_DETAIL 데이터 ② O004 ~ O007
INSERT INTO ORDER_DETAIL VALUES('OD019', 'O004', 'P043', 1, 2480);
INSERT INTO ORDER_DETAIL VALUES('OD020', 'O004', 'P044', 1, 1480);
INSERT INTO ORDER_DETAIL VALUES('OD021', 'O004', 'P046', 1, 6900);
INSERT INTO ORDER_DETAIL VALUES('OD022', 'O004', 'P050', 1, 3480);
INSERT INTO ORDER_DETAIL VALUES('OD023', 'O005', 'P001', 1, 4990);
INSERT INTO ORDER_DETAIL VALUES('OD024', 'O005', 'P008', 1, 5990);
INSERT INTO ORDER_DETAIL VALUES('OD025', 'O005', 'P028', 1, 21200);
INSERT INTO ORDER_DETAIL VALUES('OD026', 'O005', 'P029', 1, 21000);
INSERT INTO ORDER_DETAIL VALUES('OD027', 'O006', 'P031', 1, 34900);
INSERT INTO ORDER_DETAIL VALUES('OD028', 'O007', 'P001', 1, 4990);
INSERT INTO ORDER_DETAIL VALUES('OD029', 'O007', 'P005', 1, 4490);
INSERT INTO ORDER_DETAIL VALUES('OD030', 'O007', 'P006', 1, 6490);
INSERT INTO ORDER_DETAIL VALUES('OD031', 'O007', 'P010', 1, 1590);
INSERT INTO ORDER_DETAIL VALUES('OD032', 'O007', 'P011', 1, 14900);
```

데이터 삽입 ⑧ ORDER_DETAIL OD033 ~ OD041

```
-- ORDER_DETAIL 데이터 ③ O008 ~ O010
INSERT INTO ORDER_DETAIL VALUES('OD033', 'O008', 'P002', 1, 4990);
INSERT INTO ORDER_DETAIL VALUES('OD034', 'O008', 'P016', 1, 12700);
INSERT INTO ORDER_DETAIL VALUES('OD035', 'O008', 'P020', 1, 5890);
INSERT INTO ORDER_DETAIL VALUES('OD036', 'O009', 'P013', 1, 11900);
INSERT INTO ORDER_DETAIL VALUES('OD037', 'O009', 'P017', 1, 8400);
INSERT INTO ORDER_DETAIL VALUES('OD038', 'O009', 'P018', 1, 15800);
INSERT INTO ORDER_DETAIL VALUES('OD039', 'O010', 'P045', 1, 7900);
INSERT INTO ORDER_DETAIL VALUES('OD040', 'O010', 'P046', 1, 6900);
INSERT INTO ORDER_DETAIL VALUES('OD041', 'O010', 'P047', 1, 9900);
```

데이터 삽입 ⑨ DELIVERY D001 ~ D005

```sql
-- DELIVERY 데이터 ① D001 ~ D005
INSERT INTO DELIVERY VALUES('D001', 'O001', '배송준비중', '컬리넥스트마일', 'KLY260720001',
  '서울특별시 강남구 테헤란로 152, 한빛아파트 101동 1203호');
INSERT INTO DELIVERY VALUES('D002', 'O002', '배송중', '컬리넥스트마일', 'KLY260720002',
  '서울특별시 송파구 올림픽로 245, 새봄아파트 104동 802호');
INSERT INTO DELIVERY VALUES('D003', 'O003', '배송완료', '컬리넥스트마일', 'KLY260720003',
  '서울특별시 마포구 월드컵북로 88, 한울빌라 302호');
INSERT INTO DELIVERY VALUES('D004', 'O004', '배송중', '컬리넥스트마일', 'KLY260720004', '경기도
  성남시 분당구 판교역로 166, 판교푸른마을 705동 1102호');
INSERT INTO DELIVERY VALUES('D005', 'O005', '배송준비중', '컬리넥스트마일', 'KLY260720005',
  '경기도 수원시 영통구 광교중앙로 145, 광교센트럴아파트 203동 901호');
```

데이터 삽입 ⑩ DELIVERY D006 ~ D010

```
-- DELIVERY 데이터 ② D006 ~ D010
INSERT INTO DELIVERY VALUES('D006', 'O006', '배송완료', '컬리넥스트마일', 'KLY260720006',
  '경기도 고양시 일산동구 중앙로 1205, 일산하늘마을 508동 403호');
INSERT INTO DELIVERY VALUES('D007', 'O007', '배송중', '컬리넥스트마일', 'KLY260720007',
  '인천광역시 연수구 송도과학로 32, 송도미래아파트 110동 1504호');
INSERT INTO DELIVERY VALUES('D008', 'O008', '배송준비중', '컬리넥스트마일', 'KLY260720008',
  '대전광역시 서구 둔산중로 54, 둔산드림아파트 306동 1001호');
INSERT INTO DELIVERY VALUES('D009', 'O009', '배송완료', '컬리넥스트마일', 'KLY260720009',
  '세종특별자치시 한누리대로 2130, 새롬가온마을 412동 704호');
INSERT INTO DELIVERY VALUES('D010', 'O010', '배송중', '컬리넥스트마일', 'KLY260720010',
  '충청남도 천안시 서북구 불당25로 176, 불당호반마을 709동 1202호');
```

기초 질의문 결과

기초 질의문

```sql
-- 전체 회원 수 조회
SELECT COUNT(*) AS MEMBER_COUNT
FROM MEMBER;

-- 재고 100개 이상 상품 조회
SELECT P_ID, P_NAME, PRICE, STOCK
FROM PRODUCT
WHERE STOCK >= 100;

-- 주문별 배송 상태 조회
SELECT ORDER_ID, DELI_STATUS
FROM DELIVERY;

-- M001 회원의 주문내역 확인
SELECT * FROM ORDERS
WHERE M_ID = 'M001';

-- 주문 금액이 10000원 이상 30000원 이하인 주문 조회
SELECT * FROM ORDERS
WHERE TOTAL_PRICE BETWEEN 10000 AND 30000;

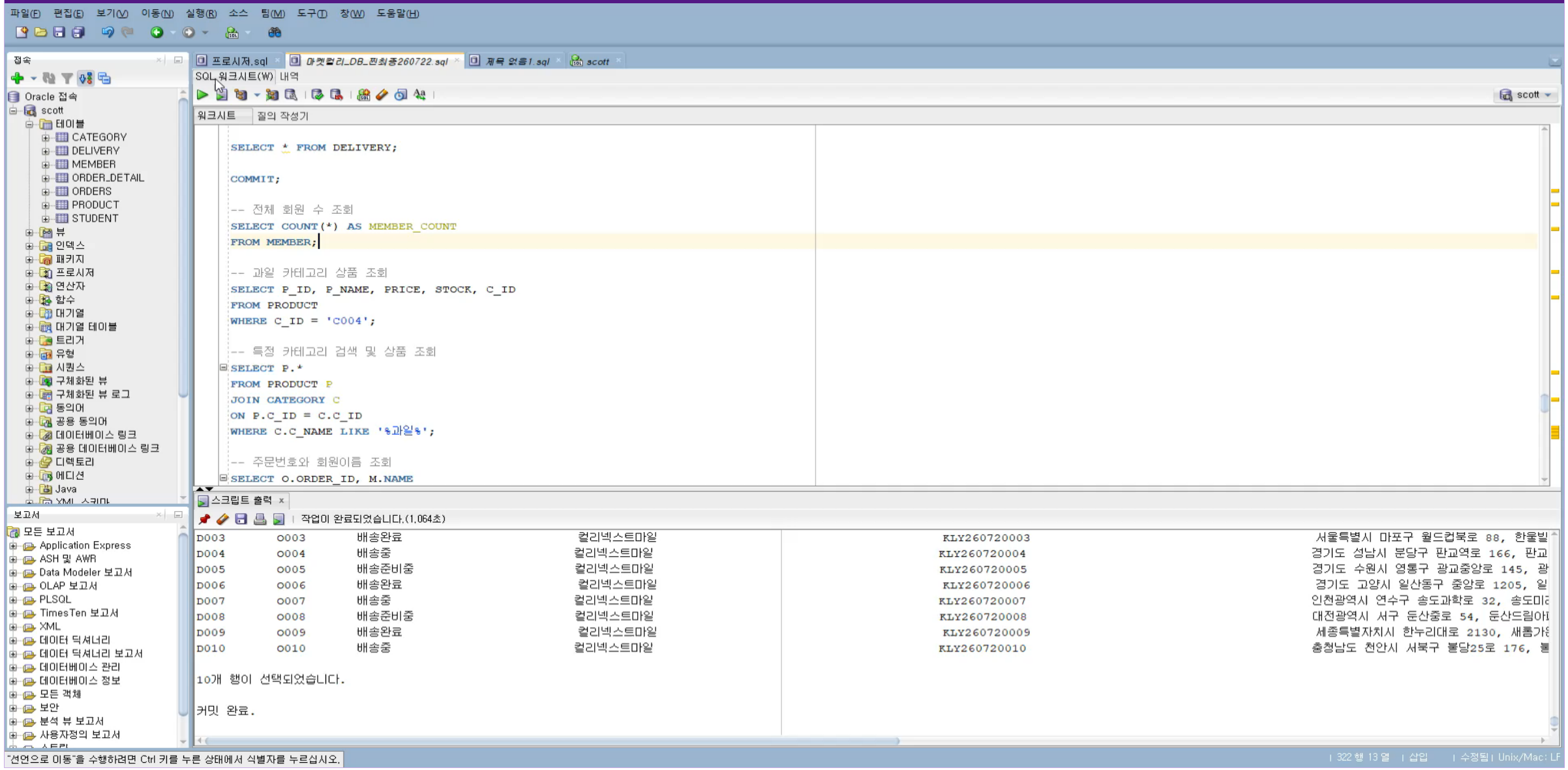
-- 최근 주문 순으로 조회
SELECT * FROM ORDERS
ORDER BY ORDER_DATE DESC;
```

실행 결과 표

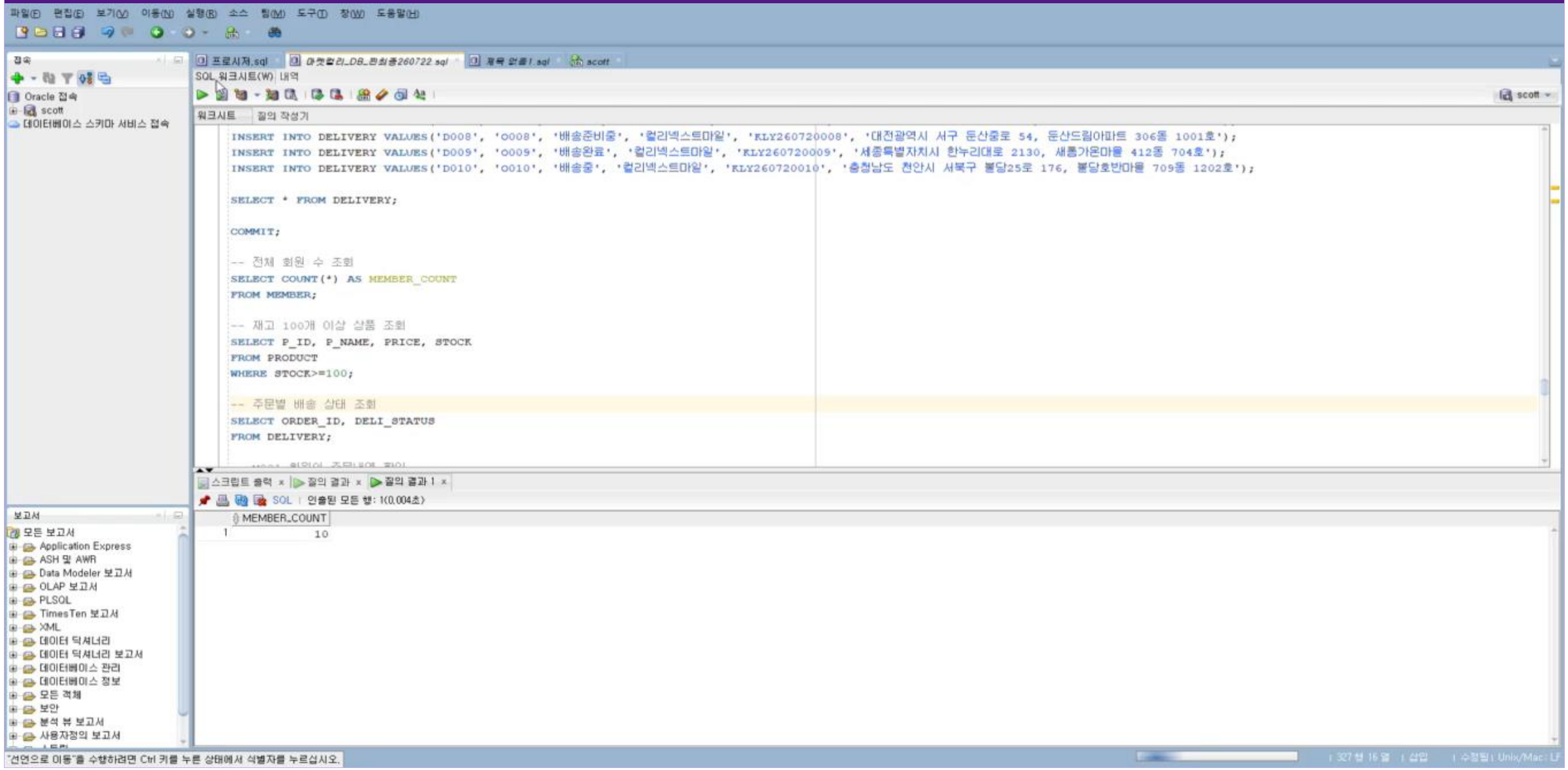
질의 내용	사용 문법	실행 결과
전체 회원 수 조회	COUNT	MEMBER_COUNT = 10
재고 100개 이상 상품	WHERE	5건: P001, P004, P010, P043, P044
주문별 배송 상태	SELECT	배송준비중·배송중·배송완료 확인
M001 회원 주문	WHERE	O001 / 2026-06-30 / 47,520원
1만~3만원 주문	BETWEEN	3건: O004, O008, O010
최근 주문 조회	ORDER BY DESC	O010 → O009 → O007/O008 순

같은 날짜의 주문은 ORDER_ID 조건을 추가하면 출력 순서를 고정할 수 있습니다.

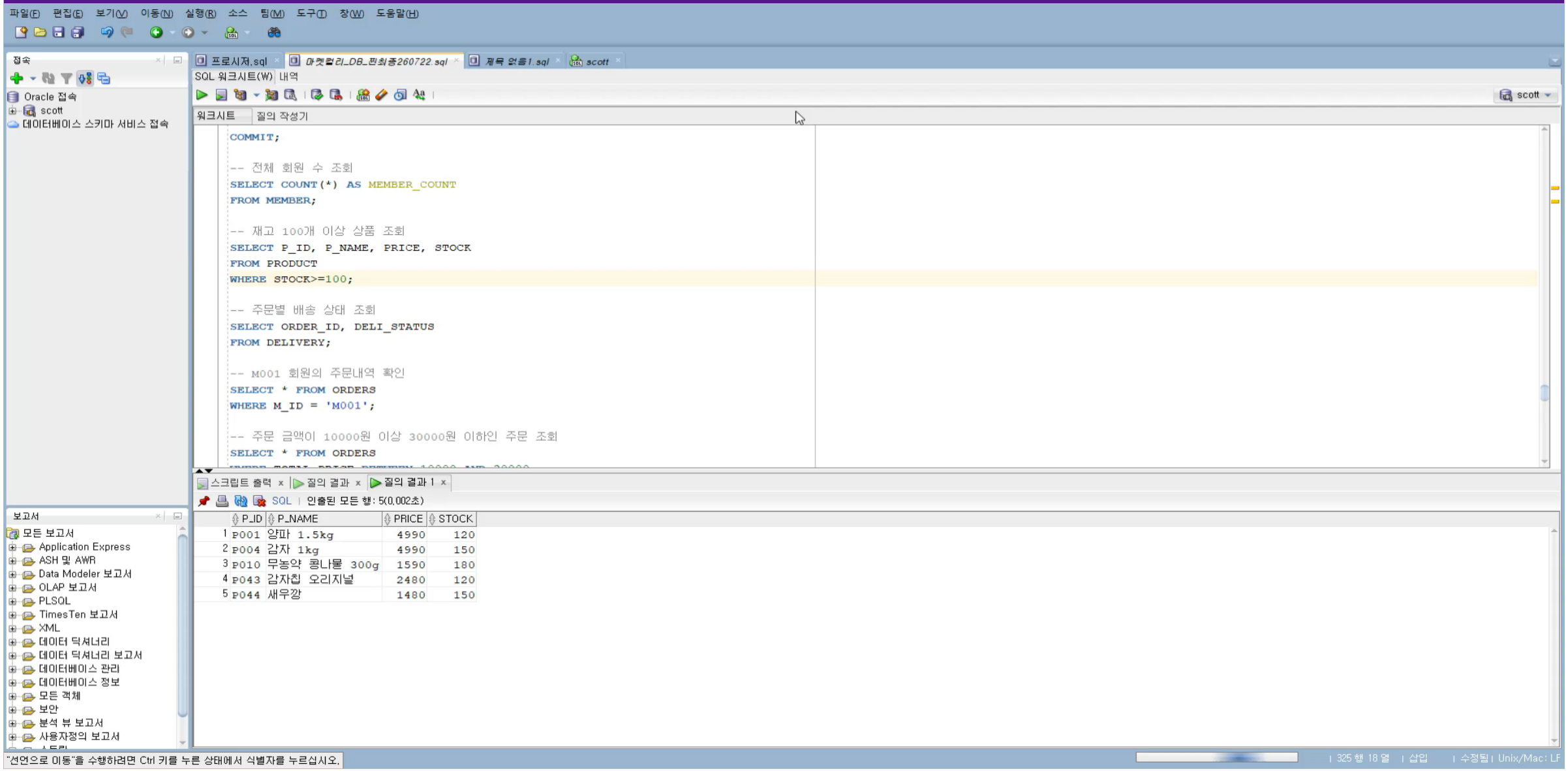
기초 질의문 결과 (전체 회원 수 조회)



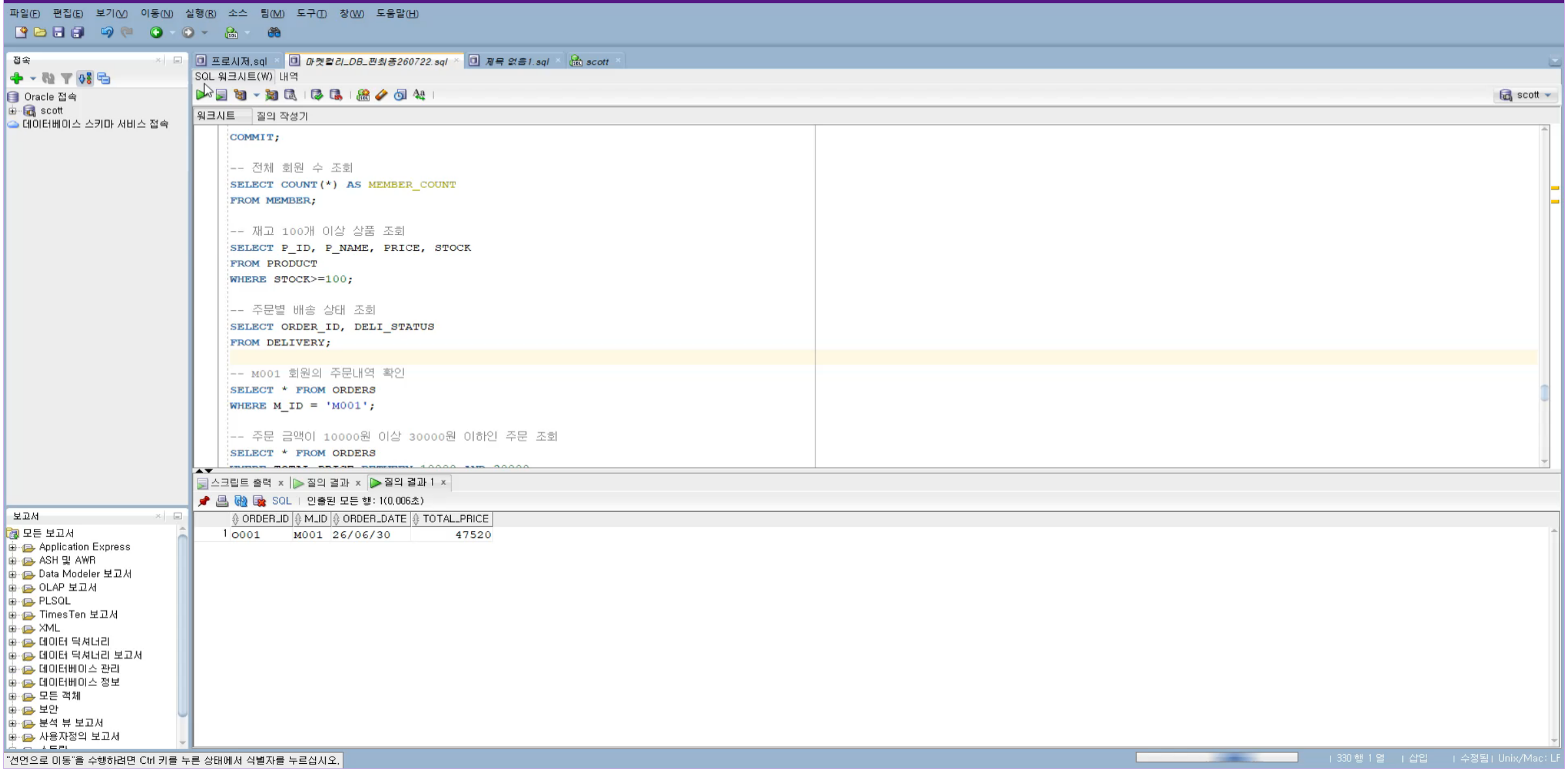
기초 질의문 결과 (재고 100개 이상 상품 조회)



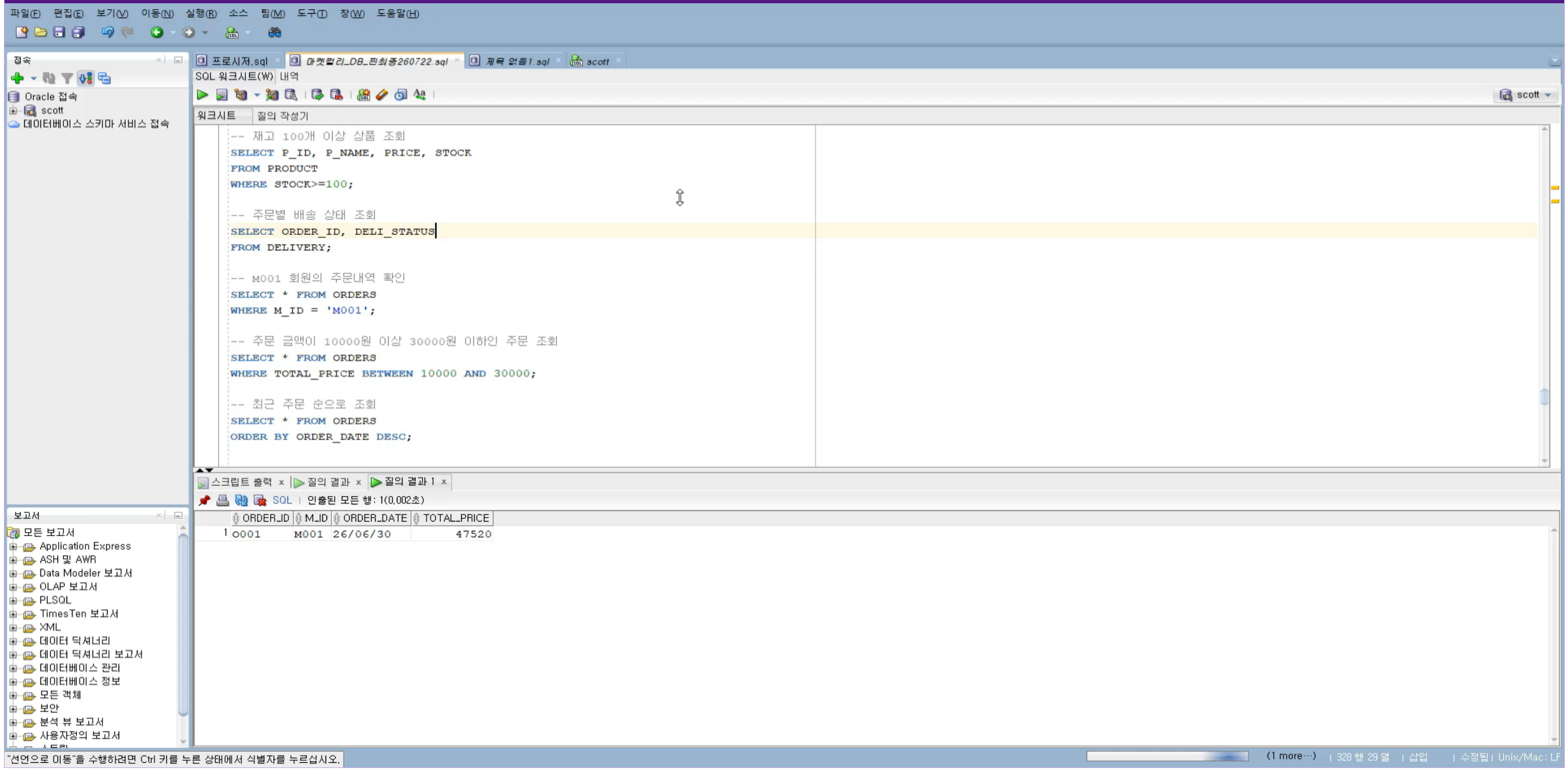
기초 질의문 결과 (주문별 배송 상태 조회)



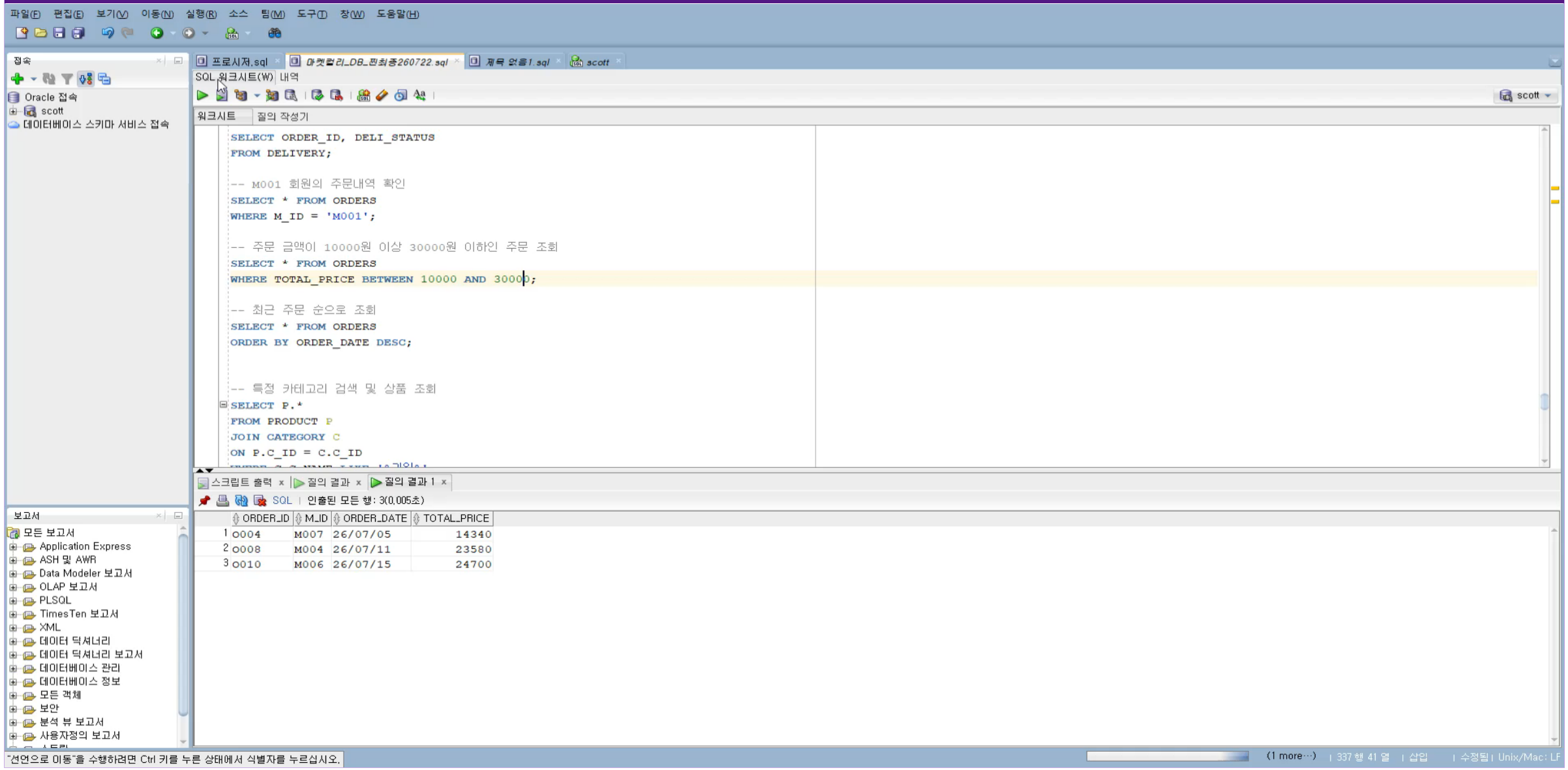
기초 질의문 결과 (M001 회원의 주문내역 조회)



기초 질의문 결과 (주문 금액이 10000원 이상 30000원 이하인 주문 조회)



기초 질의문 결과 (최근 주문순으로 조회)



응용 질의문 결과

응용 질의문 JOIN · GROUP BY · 서브쿼리

```sql
-- 과일·견과·쌀 카테고리 검색 및 상품 조회
SELECT P.P_ID, P.P_NAME, P.PRICE,
    P.STOCK, C.C_NAME AS CATEGORY_NAME
FROM PRODUCT P
JOIN CATEGORY C ON P.C_ID = C.C_ID
WHERE C.C_NAME LIKE '%과일%';
-- 주문번호와 회원이름 조회
SELECT O.ORDER_ID, M.NAME
FROM ORDERS O
JOIN MEMBER M ON O.M_ID = M.M_ID;

-- 회원별 총 주문 금액 확인(GROUP BY)
SELECT M_ID, SUM(TOTAL_PRICE) AS TOTAL_AMOUNT
FROM ORDERS
GROUP BY M_ID;

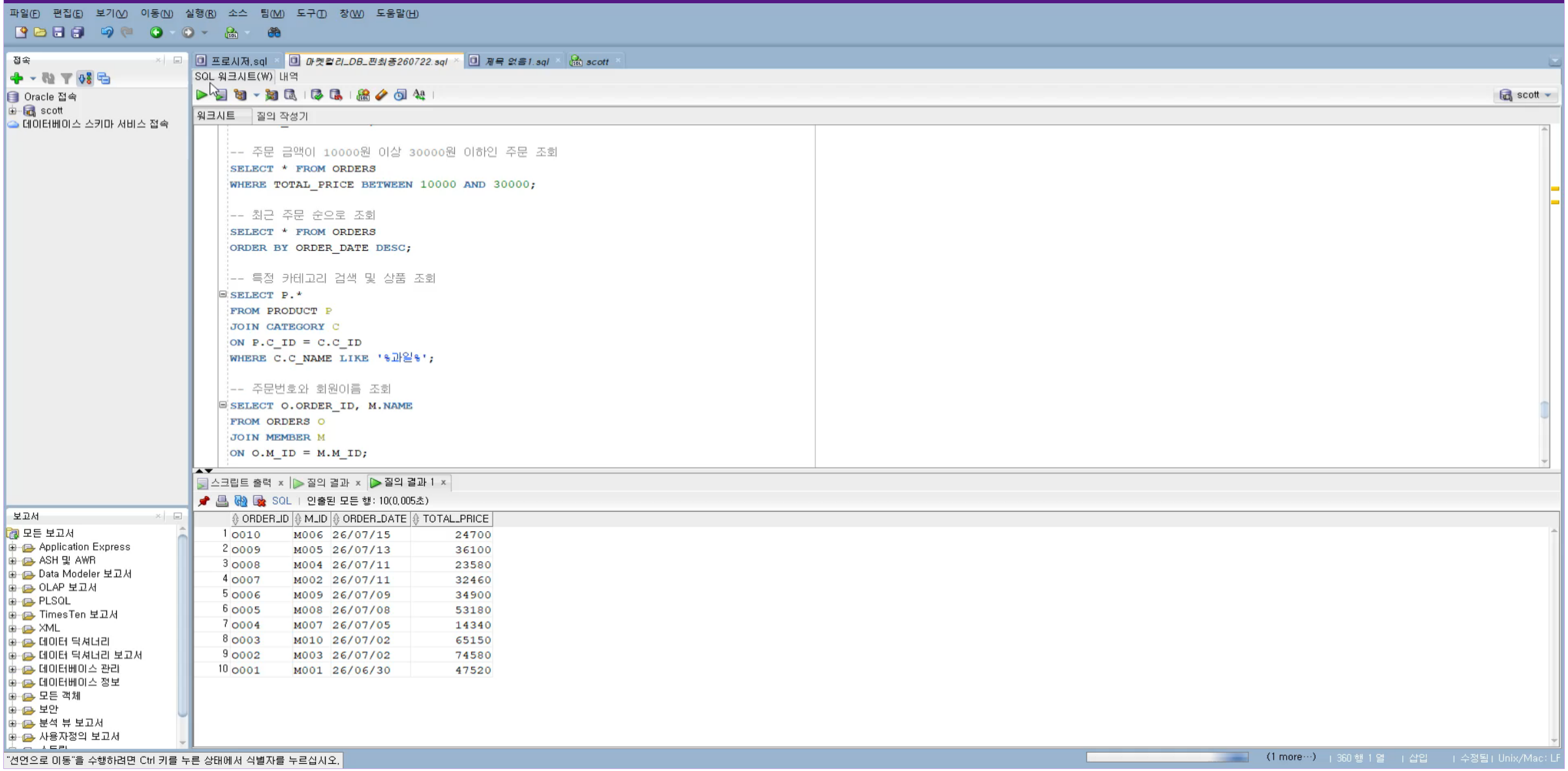
-- 가장 비싼 상품정보 (서브쿼리)
SELECT P_NAME, PRICE
FROM PRODUCT
WHERE PRICE = (SELECT MAX(PRICE) FROM PRODUCT);
```

실행 결과 표

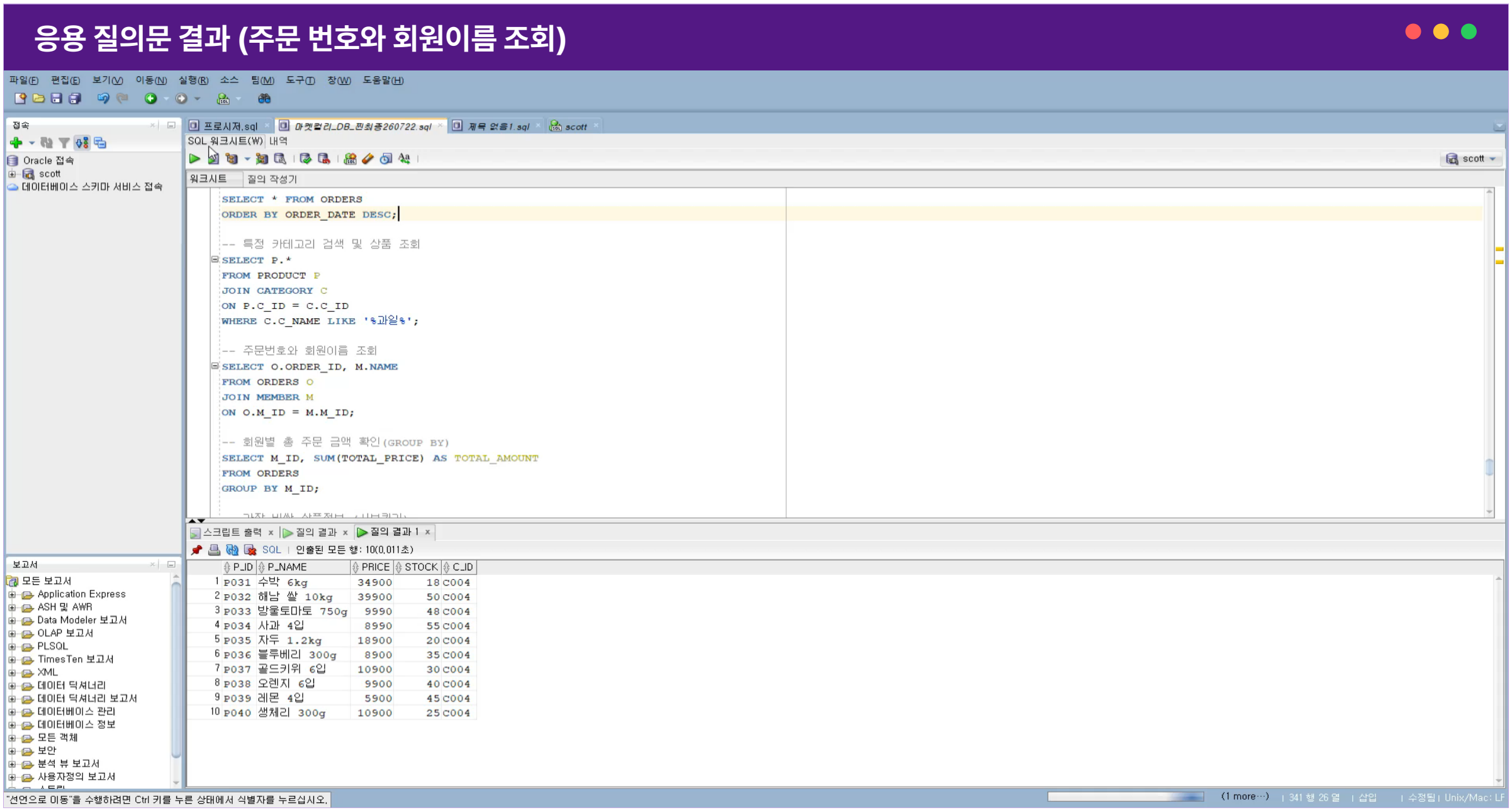
질의 내용	사용 문법	실행 결과
카테고리 포함 상품 조회	JOIN + LIKE	10건 / C_NAME = 과일, 견과, 쌀
주문번호와 회원명 조회	JOIN	O001 김민준, O002 박지훈, O003 송하은 등
회원별 총 주문금액	GROUP BY	10건 / M003 = 74,580원으로 최고
가장 비싼 상품	서브쿼리	해남 쌀 10kg / 39,900원

카테고리명까지 보이도록 SELECT에 C.C_NAME AS CATEGORY_NAME을 포함했습니다.

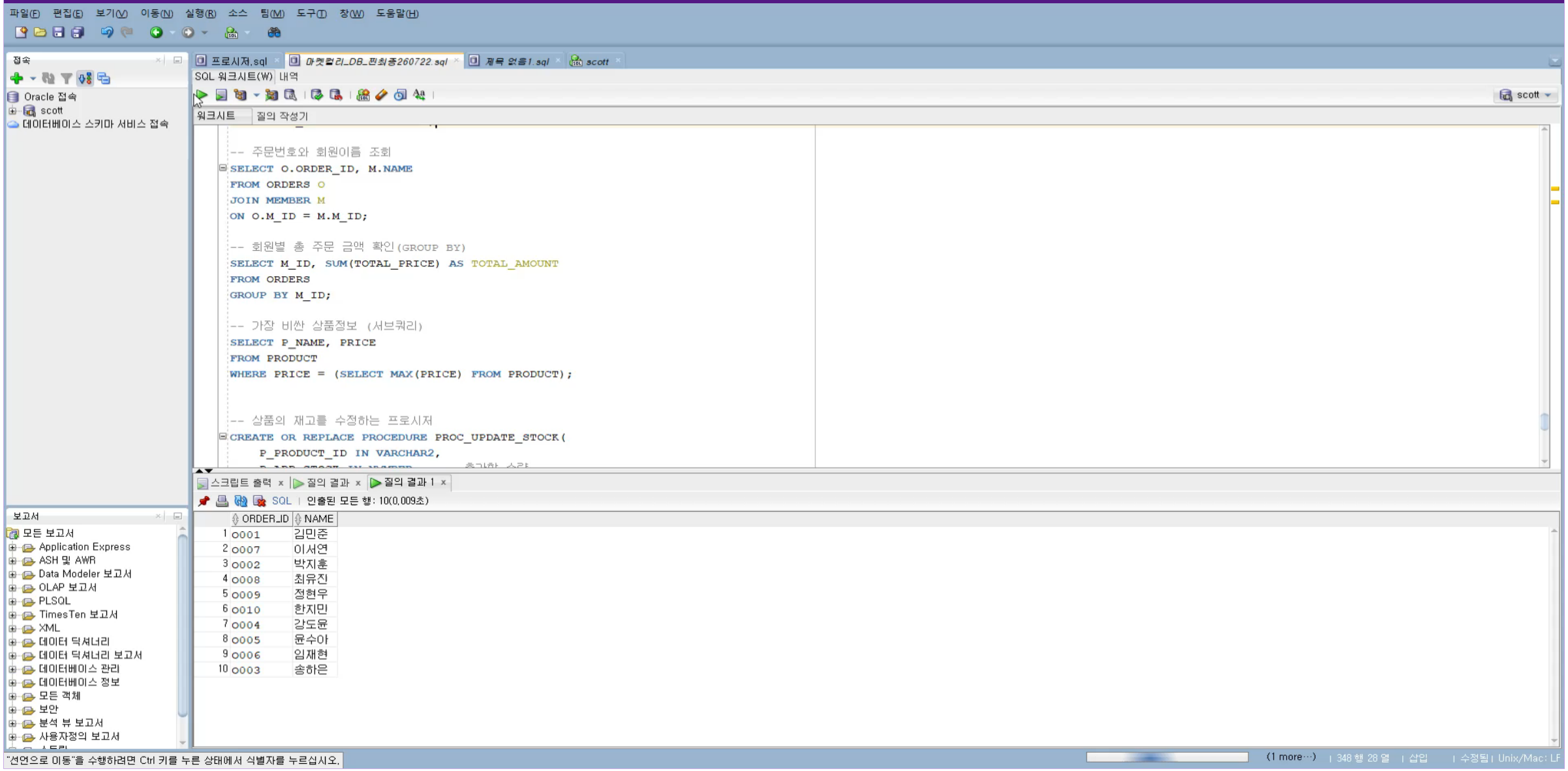
응용 질의문 결과 (과일, 견과, 쌀 카테고리 검색 및 상품 조회)



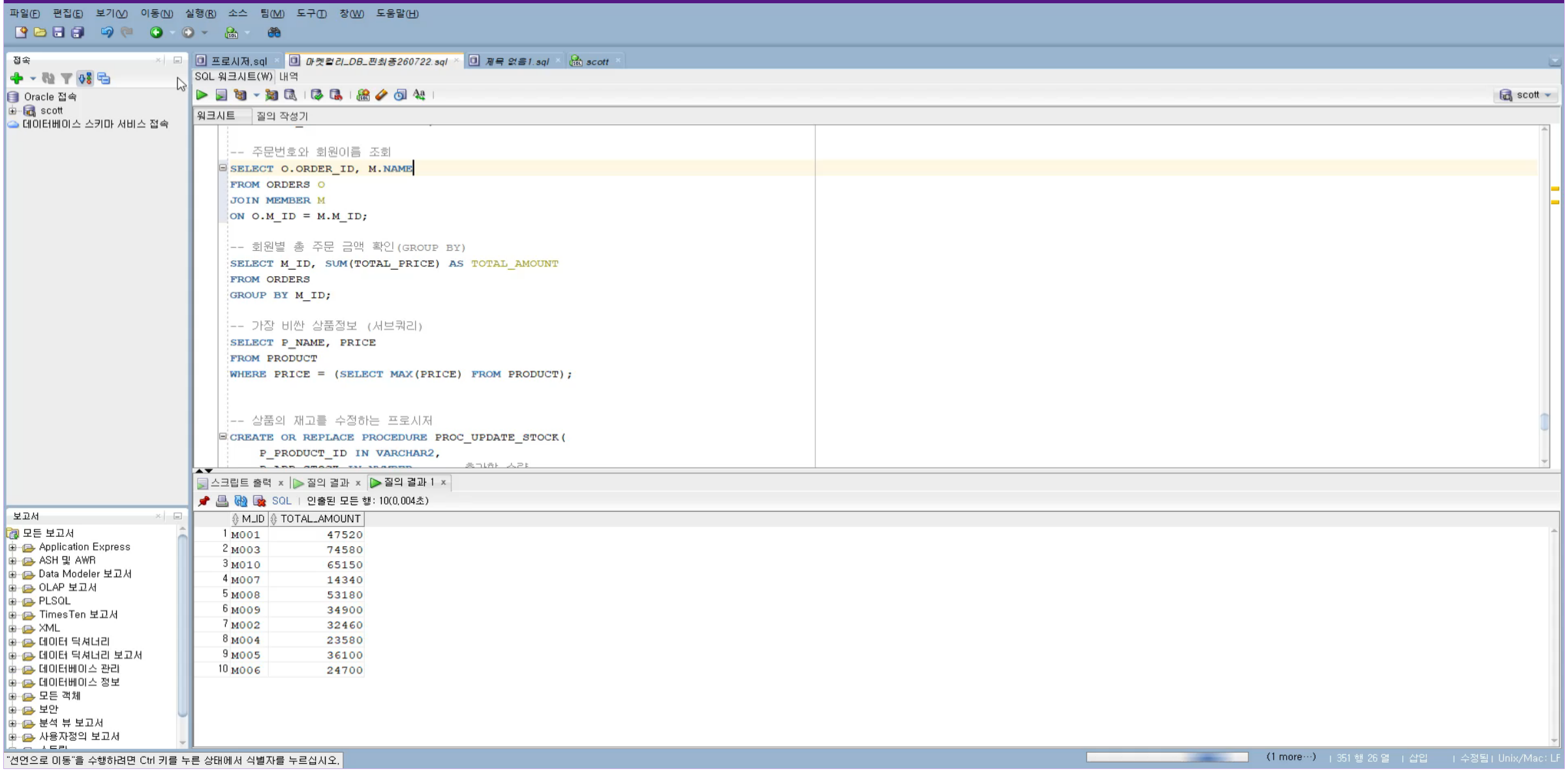
응용 질의문 결과 (주문 번호와 회원이름 조회)



응용 질의문 결과 (회원 별 총 주문 금액 확인)



응용 질의문 결과 (가장 비싼 상품정보 (서브쿼리))



프로시저 생성: 재고 추가 기능

```
01  CREATE OR REPLACE PROCEDURE PROC_UPDATE_STOCK(
02      P_PRODUCT_ID IN VARCHAR2,
03      P_ADD_STOCK  IN NUMBER    -- 추가할 수량
04  )
05  IS
06      V_BEFORE_STOCK NUMBER;
07  BEGIN
08      -- 1. 변경 전 현재 재고 조회
09      SELECT STOCK
10        INTO V_BEFORE_STOCK
11        FROM PRODUCT
12       WHERE P_ID = P_PRODUCT_ID;
13
14      -- 2. 전달받은 수량만큼 재고 추가
15      UPDATE PRODUCT
16         SET STOCK = V_BEFORE_STOCK + P_ADD_STOCK
17       WHERE P_ID = P_PRODUCT_ID;
18
19      COMMIT;
20  END;
21  /
```

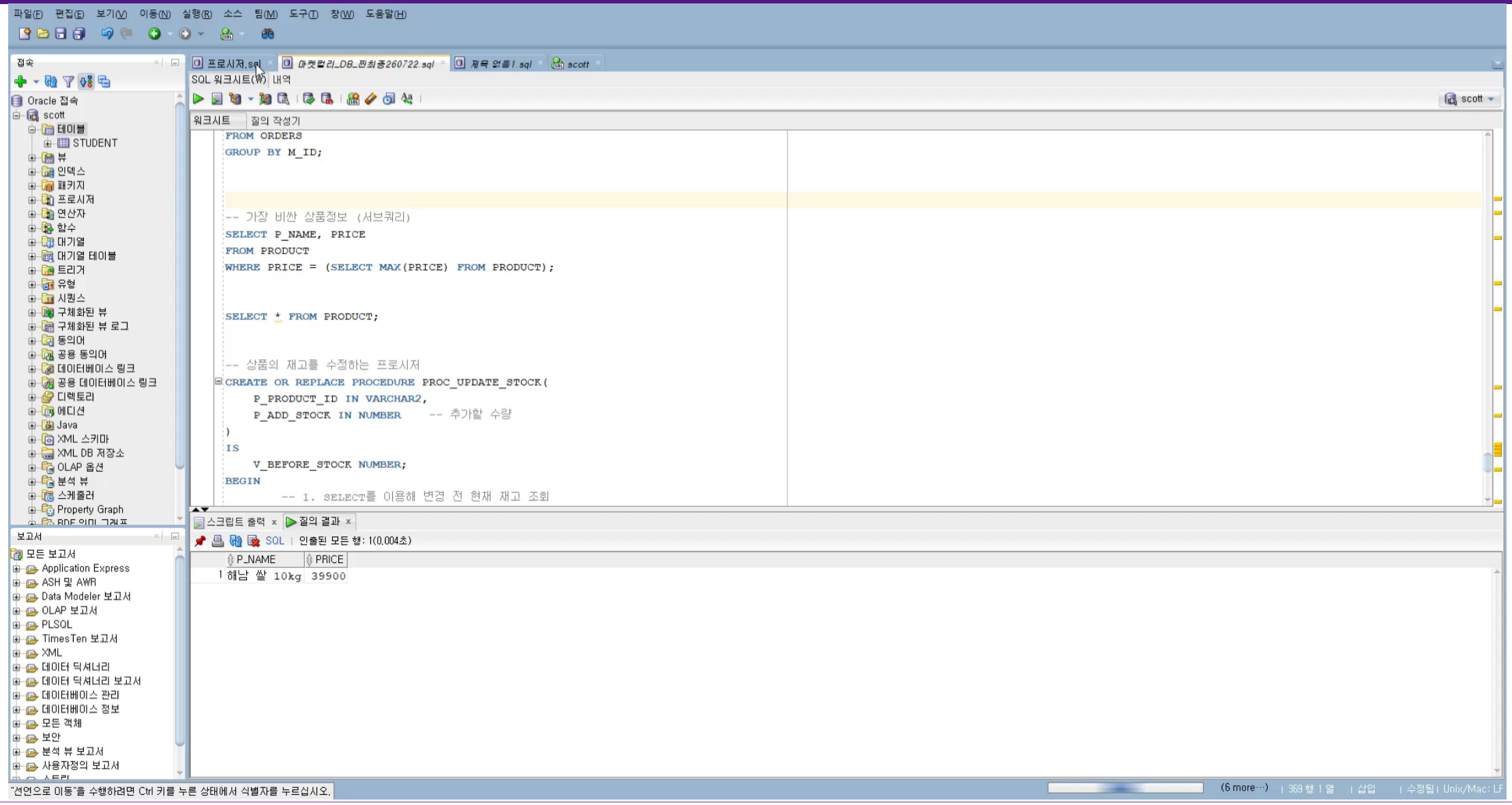
실행 결과

Procedure PROC_UPDATE_STOCK created.

입력값	상품번호 + 추가수량
현재 재고 조회	SELECT STOCK INTO 변수
재고 수정	기존 재고 + 추가수량
저장 처리	COMMIT으로 변경 저장

현재 SQL 기준: 오류 처리 없이
정상 재고 추가 기능을 구현

프로시저 실행 영상


파일(F) 편집(E) 보기(V) 이동(N) 실행(R) 소스 팀(M) 도구(T) 창(W) 도움말(H)
프로시저.sql
마켓컬리_DB_민최종260722.sql
제목 없음1.sql
scott
SQL 워크시트(W) 내역
워크시트 질의 작성기
FROM ORDERS
GROUP BY M_ID;
-- 가장 비싼 상품정보 (서브쿼리)
SELECT P_NAME, PRICE
FROM PRODUCT
WHERE PRICE = (SELECT MAX(PRICE) FROM PRODUCT);
SELECT * FROM PRODUCT;
-- 상품의 재고를 수정하는 프로시저
CREATE OR REPLACE PROCEDURE PROC_UPDATE_STOCK(
P_PRODUCT_ID IN VARCHAR2,
P_ADD_STOCK IN NUMBER -- 추가할 수량
)
IS
V_BEFORE_STOCK NUMBER;
BEGIN
-- 1. SELECT를 이용해 변경 전 현재 재고 조회
스크립트 출력
질의 결과
SQL | 인출된 모든 행: 1(0.004초)
P_NAME PRICE
1 해남 쌀 10kg 39900
Oracle 접속
scott
테이블
STUDENT
뷰
인덱스
패키지
프로시저
연산자
함수
대기열
대기열 테이블
트리거
유형
시퀀스
구체화된 뷰
구체화된 뷰 로그
동의어
공용 동의어
데이터베이스 링크
공용 데이터베이스 링크
디렉토리
에디션
Java
XML 스키마
XML DB 저장소
OLAP 옵션
분석 뷰
스케줄러
Property Graph
보고서
모든 보고서
Application Express
ASH 및 AWR
Data Modeler 보고서
OLAP 보고서
PLSQL
TimesTen 보고서
XML
데이터 딕셔너리
데이터 딕셔너리 보고서
데이터베이스 관리
데이터베이스 정보
모든 객체
보안
분석 뷰 보고서
사용자정의 보고서
"선언으로 이동"을 수행하려면 Ctrl 키를 누른 상태에서 식별자를 누르십시오.
(6 more···) | 369 행 1 열 | 삽입 | 수정됨 | Unix/Mac: LF

프로시저 실행 결과

프로시저 실행 및 결과 확인

```
-- 1. P001 재고를 초기값(120개)으로 원복
UPDATE PRODUCT SET STOCK = 120 WHERE P_ID = 'P001';
COMMIT;

-- 2. 변경 전 확인 -> 결과: 120개
SELECT P_ID, P_NAME, STOCK
FROM PRODUCT
WHERE P_ID = 'P001';

-- 3. 프로시저 실행 -> P001 재고 100개 추가
EXEC PROC_UPDATE_STOCK('P001', 100);

-- 4. 변경 후 확인 -> 결과: 220개
SELECT P_ID, P_NAME, STOCK
FROM PRODUCT
WHERE P_ID = 'P001';
```

구분	실행 내용 / 결과
변경 전	P001 / 양파 1.5kg / STOCK 120
실행문	EXEC PROC_UPDATE_STOCK('P001', 100);
처리 결과	PL/SQL procedure successfully completed.
변경 후	P001 / 양파 1.5kg / STOCK 220
의미	재고가 100개 증가함: 120 → 220

실행 전 SELECT → EXEC → 실행 후 SELECT 순서로 재고 변경 결과를 확인했습니다.

개인별 평가 소감

팀장/팀원별 프로젝트를 통해 배운 점

이름	개인 평가 소감
박슬지	DB를 직접 구축해 보며 전반적인 작성 과정과 흐름을 알 수 있는 유익한 시간이었습니다. 특히 미니 프로젝트를 통해 이론으로만 접하던 DB 생성 절차가 실제로 어떻게 기획되고 적용되는지 직접 경험해 볼 수 있었습니다.
민재홍	이번 프로젝트를 진행하며 AI를 활용해 작업의 효율을 높일 수 있었지만, 오류를 찾고 테이블 간 관계를 이해하려면 결국 기본적인 데이터베이스 지식이 필요하다는 것을 느꼈습니다. AI 시대일수록 결과를 그대로 받아들이기보다 직접 확인하고 판단하는 능력이 중요하다고 생각했습니다.
장영훈	강의로만 듣고 배우던 DB설계와 SQL을 프로젝트와 연계해보며 경험을 쌓아볼 수 있는 좋은 기회였습니다.
이남규	이번 팀 프로젝트에서 SQL Developer를 활용해 마켓컬리를 주제로 DB를 설계하고 구현해보았는데 실무적인 경험을 쌓을 수 있는 좋은 기회였습니다.
손지혜	이론 수업만 들었을 때는 SQL의 개념을 어느 정도 이해하는 하고 있었지만 실제 프로젝트를 진행하며 직접 SQL을 작성하고 실행해 보니 SQL의 용어와 핵심 개념을 더욱 자세하게 이해할 수 있었습니다.

감사합니다

마켓컬리 주문 및 배송 관리 시스템



질문 있으시면 답변드리겠습니다.